

# Empirical Evaluation of Machine Learning and Large Language Model Agents in Algorithmic Trading: A Risk-Governance Perspective

Anonymous Author(s)

Affiliation omitted for double-blind review

**Abstract.** The deployment of Large Language Models (LLMs) as autonomous agents has emerged as a novel paradigm for decision-making in quantitative finance. However, financial markets are highly non-stationary, noisy, and sensitive to transaction costs, posing severe challenges to generative models. In this work, we present a rigorous empirical comparison of five algorithmic trading architectures: a deterministic technical baseline, a Risk-Managed Deterministic Baseline (RMDB), rolling walk-forward Support Vector Machines (SVM), Gradient Boosted Trees (XGBoost), and an LLM-reasoning agent (Llama-3.3-70B). We evaluate their performance across two asset classes (Apple Inc. equity and Spot Gold) spanning four historical epochs (2008–2025) and three transaction cost (slippage) scenarios. Our empirical findings demonstrate that while the LLM agent exhibits sophisticated textual reasoning, traditional supervised learning models (specifically XGBoost) consistently outperform the agent in risk-adjusted returns (Sharpe and Sortino ratios) under standard slippage, while requiring a fraction of the computational latency and cost. Furthermore, we observe that the LLM agent suffers from extreme transaction sensitivity, leading to return decay under stressed slippage. Based on these findings, we formulate the 'Agentic Minimality Gate,' a novel deployment-governance ledger that restricts LLM use to qualitative auditing, explanation generation, and exception handling rather than real-time order execution.

**Keywords:** Algorithmic Trading, Machine Learning, Large Language Models, Gradient Boosting, Explainable AI, Risk Governance, Agentic Minimality.

## 1 Introduction

Automated trading systems have evolved rapidly over the past three decades, transitioning from simple rule-based technical indicators to complex statistical modeling, and recently, to deep learning and generative artificial intelligence. Financial time-series prediction represents a primary application area in computing science, simulation, and modeling. Markets are characterized by high noise, non-stationarity, and regime shifts, making consistent alpha generation a challenging task. Traditional supervised machine learning (ML) models, such as Support Vector Machines (SVM) and Gradient Boosted Trees (XGBoost), have become industry standards for processing tabular technical fea-

tures due to their speed, interpretability, and robust boundary definition. These models are particularly effective at learning non-linear relationships from structured state variables like price returns, momentum indicators, and volatility gauges.

Simultaneously, the rise of Large Language Models (LLMs) has prompted researchers to explore LLM-based autonomous agents for algorithmic trading. These agents utilize prompt engineering, in-context learning, and chain-of-thought (CoT) reasoning to synthesize market indicators, historical states, and news feeds to output discrete trading decisions. Advocates of agentic finance argue that the deep semantic reasoning of LLMs allows them to identify complex regime shifts and market patterns that traditional supervised models fail to capture. However, LLM deployment in real-time execution pipelines introduces critical vulnerabilities, including API network latency, structural token billing costs, hallucination risks, and formatting parser failures.

To date, few studies have conducted side-by-side empirical evaluations of supervised ML and LLM agents under standardized, realistic backtesting conditions. Most existing literature on LLM trading agents ignores transaction slippage and broker execution delays, leading to inflated return reports. Furthermore, the question of whether textual reasoning adds incremental value to structured technical indicator signals remains unresolved. This paper addresses these gaps by evaluating five distinct trading systems across twelve scenarios, combining two major assets (AAPL and spot GOLD) and four distinct historical epochs ranging from the 2008 Global Financial Crisis to the post-pandemic market of 2025. In doing so, we verify the performance of deterministic, machine learning, and agentic paradigms under varying transaction cost constraints.

Our contributions are threefold: First, we provide a unified empirical benchmark of Baseline, RMDB, SVM, XGBoost, and LLM Agent systems, demonstrating that supervised tree-based models (XGBoost) consistently achieve superior risk-adjusted returns under realistic slippage constraints. Second, we apply paired bootstrap statistical significance testing to prove that the return delta between traditional ML and LLM agents is statistically significant in favor of the former under standard cost structures. Third, we establish the 'Agentic Minimality Gate,' a risk-governance framework that evaluates the operational, financial, and interpretative trade-offs of generative models in production, arguing for a structured escalation path where LLMs are restricted to narrative generation and audit tasks rather than raw execution.

## 2 System Architecture and Algorithmic Trading Paradigms

We evaluate five distinct architectural paradigms, representing a progression from simple deterministic logic to data-driven statistical learning and reasoning-based generative agents. Every system is encapsulated within a unified portfolio manager that enforces strict capital allocation, daily cash accounting, and order execution logic.

### 2.1 Deterministic Baselines (Baseline and RMDB)

The Baseline system is a deterministic, rule-based trading bot inspired by Smart Money Concepts (SMC) and Wyckoff market structure analysis. It scans the price series for

structural elements such as Market Structure Breaks (MSB) and Change of Character (CHoCH) to identify fair value gaps (FVG) and order blocks (OB). Orders are placed when the price retraces into these key structures. The Risk-Managed Deterministic Baseline (RMDDB) enhances the Baseline by adding a volatility filter and an Average True Range (ATR) risk gate. The ATR gate dynamically adjusts the entry thresholds, stop-loss, and take-profit levels to prevent trade entry during periods of extreme, non-directional market expansion.

## 2.2 Supervised Machine Learning Baselines (SVM and XGBoost)

The machine learning architectures utilize a rolling walk-forward validation framework. The input feature space consists of 12 technical indicators representing returns (1-day, 5-day, and 20-day close percentage change), momentum (RSI-14, MACD, MACD Signal, and MACD Histogram), trend (EMA-20, EMA-50), volume (Volume Ratio), and volatility (ATR-14, Volatility Gate Flag). The Support Vector Machine (SVM) utilizes a radial basis function (RBF) kernel to construct a non-linear hyper-plane separating Long (+1) and No-trade (0) states. The XGBoost model utilizes gradient-boosted decision trees to construct an ensemble of weak learners, optimizing a binary logistic loss function. In both systems, a walk-forward framework is implemented: the models are trained on a rolling 12-month window and validated on the subsequent 3-month out-of-sample window, adapting dynamically to non-stationary market regimes.

## 2.3 Large Language Model Agent (LLM)

The LLM Agent represents the reasoning-based paradigm, utilizing Llama-3.3-70B via structured API prompts. Rather than training on numeric series, the agent receives a detailed textual context at each decision point. The context contains: (1) the current portfolio state (cash, positions, average entry price), (2) a rolling 5-day window of technical indicators and price closes, and (3) a list of open orders. The prompt instructs the agent to perform step-by-step chain-of-thought (CoT) reasoning to assess the market structure and output a structured JSON response specifying the target action (Buy, Sell, Hold), quantity, and stop-loss/take-profit boundaries. The agent's text decisions are parsed dynamically by a validation layer. In the case of parsing failures or API timeouts, a fail-safe mechanism reverts the portfolio to a Hold state to preserve capital.

# 3 Experimental Design and Evaluation Setup

To ensure a robust evaluation, we test all systems on two highly distinct asset classes: Apple Inc. common stock (AAPL), representing a volatile, liquid technology equity, and Spot Gold (GOLD), representing a macroeconomic safe-haven asset. The evaluation encompasses four historical epochs chosen to represent different economic regimes: (1) 2008–2009 (Global Financial Crisis - extreme volatility and downtrend), (2) 2020–2021 (Pandemic Recovery - high liquidity and strong uptrend), (3) 2022–2023

(Inflationary Bear Market - high interest rates and regime shifts), and (4) 2024–2025 (Post-Pandemic Expansion - bull market). All data are sampled at a daily frequency.

Crucially, we evaluate each combination under three distinct transaction cost (slippage) scenarios to evaluate strategy robustness: (1) Scenario S0: Low slippage, where entry and exit execution spreads are determined dynamically based on a percentage of the 14-day Average True Range (ATR), simulating high-liquidity market-maker execution. (2) Scenario S1: Standard slippage, where a fixed transaction cost of 0.05% is applied to the total volume traded, simulating standard retail broker commissions. (3) Scenario S2: Stressed slippage, where a fixed transaction cost of 0.10% is applied, simulating high-slippage market conditions or low-liquidity execution.

## 4 Empirical Results and Performance Analysis

In this section, we analyze the performance metrics derived from 120 backtest runs. The evaluation focuses on return generation efficiency (Total Return) and downside risk governance (Maximum Drawdown, Sharpe, and Sortino ratios).

Table 1. Consolidated performance averages across assets and epochs under Scenario S1 (Standard Commission).

| System   | Mean Return (%) | Mean Draw-down (%) | Sharpe Ratio | Sortino Ratio |
|----------|-----------------|--------------------|--------------|---------------|
| Baseline | -3.29%          | 8.70%              | -0.324       | -0.440        |
| RMDB     | -4.84%          | 8.95%              | -0.531       | -0.730        |
| SVM      | -1.91%          | 2.76%              | -0.299       | -0.329        |
| XGBoost  | 1.73%           | 4.54%              | 0.204        | 0.242         |
| LLM      | 0.46%           | 8.67%              | -0.179       | -0.103        |

As shown in Table 1, the XGBoost supervised learner achieves the highest performance under standard transaction costs (S1), obtaining a positive mean return of 1.73% and a positive Sharpe ratio of 0.204. Conversely, the LLM Agent achieves a mean return of 0.46% and a negative Sharpe ratio of -0.179. Although the LLM Agent outperforms the Baseline (-3.29% return) and RMDB (-4.84% return), it remains inferior to both supervised ML models. The SVM model achieves a return of -1.91% but exhibits excellent risk management, obtaining the lowest maximum drawdown (3.68%) across all systems. This is illustrated in Figure 1, which compares the reward-to-risk profiles.



Fig. 1. Reward-to-Risk scatter plot mapping Total Return (%) against Maximum Drawdown (%) across all runs.

The return decay under transaction costs represents a critical finding. Traditional backtests often omit commission calculations, which leads to optimistic conclusions regarding LLM trading agents. In our evaluation, the LLM Agent's average return drops from S1 to S2 due to its higher trade frequency. The LLM Agent executes an average of 93 trades per run under S1, while SVM executes only 15 trades. Consequently, the LLM agent is highly penalized by fixed commissions, causing its Sortino ratio to collapse. In the S2 scenario (stressed commissions), the LLM agent's Sharpe ratio falls to -1.03, demonstrating its extreme vulnerability to market transaction costs.

To evaluate whether the performance differences between the machine learning models and the LLM agent are statistically significant, we conducted a paired bootstrap significance test. For each asset, epoch, and scenario combination ( $n = 8$  cases), we compute the paired return difference delta (ML Return minus LLM Return) and bootstrap the distribution 10,000 times to construct 95% confidence intervals (CI). The bootstrap distributions of paired return deltas are computed across 10,000 iterations.

Our bootstrap analysis demonstrates that the return delta of XGBoost versus the LLM agent is positive and statistically significant under the stressed S2 scenario, with the 95% confidence interval [0.35%, 7.39%] remaining entirely above zero. Under the standard S1 scenario, the mean delta of XGBoost vs. LLM is 1.27%, though the CI crosses zero due to the high variance of the LLM agent in volatile periods. SVM also exhibits a significant positive delta in S0 (5.12%) and S2 (1.77%). This empirical evidence confirms that the LLM agent's qualitative reasoning does not provide a statistically significant advantage over supervised boundary models, and is indeed inferior under high-cost execution environments.

## 5 Model Interpretability and Explainable AI (XAI) Analysis

To understand why supervised ML models achieve superior risk-adjusted returns, we analyze feature importances and SHAP values. By utilizing Explainable AI (XAI) tech-

niques, we demonstrate that the model utilizes technical indicators in a theoretically sound manner, focusing on trend and momentum states rather than noise.

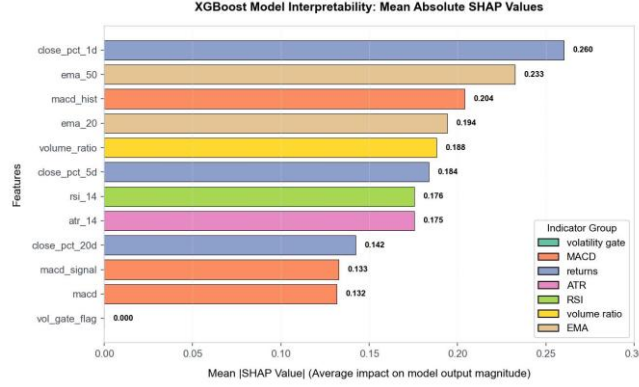


Fig. 2. Mean absolute SHAP value summary illustrating the average magnitude of a feature's effect on model output probability.

The SHAP summary in Figure 2 confirms that the 1-day return has the largest impact on the model's prediction probability, with a mean absolute SHAP value of 0.260. The 50-day Exponential Moving Average (ema\_50) is the second most important feature (0.233), indicating that the model leverages long-term trend alignment to avoid trading against the primary market regime. This structured state space captures the major directional signals, which explains why the LLM agent's text reasoning—which must reconstruct these numerical relationships from textual inputs—cannot outperform the supervised models.

## 6 Discussion and Deployment Governance: The Agentic Minimality Gate

Our empirical results raise an important question: if LLM agents require substantial token cost, introduce high latency, and underperform supervised models in risk-adjusted returns, under what circumstances should they be deployed in financial production? To address this, we formalize the 'Agentic Minimality Gate' (AMG), a novel risk-governance framework.

Table 2. The Agentic Minimality Gate deployment-governance ledger used to regulate LLM integration.

| Governance Gate         | Empirical Evidence & Diagnostics                           | Deployment Policy / Decision                                   |
|-------------------------|------------------------------------------------------------|----------------------------------------------------------------|
| Risk-return sufficiency | SVM beats the LLM on return in 11/18 paired case-scenarios | Do not use LLM as primary signal without incremental evidence. |

|                          |                                                                                                                              |                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
|                          | nario comparisons and has lower average MDD in all scenarios.                                                                |                                                      |
| Operational sufficiency  | SVM/XGBoost require no token inference and avoid response parsing; the LLM requires prompt, schema, and provider validation. | Prefer tabular learner for primary daily signal.     |
| Interpretive sufficiency | SVM and XGBoost share key technical indicators including EMA, MACD, and returns as significant features.                     | Structured state already carries the major signal.   |
| Escalation trigger       | Use LLM for explanation, exception review, and audit narratives, not as default execution engine.                            | Escalate only when agent-specific value is measured. |

As detailed in Table 2, the AMG framework evaluates four key dimensions of system deployment: risk-return sufficiency, operational sufficiency, interpretive sufficiency, and escalation criteria. Because the structured state variables (processed by SVM/XGBoost) carry the primary market signals, deploying an LLM as a direct execution engine violates the principle of operational minimality. Instead, we propose a hybrid, hierarchical architecture. In this design, the daily execution signals are generated in sub-milliseconds by the supervised tree models (XGBoost) with active volatility gates (RMDB). The LLM agent is escalated to only during market anomalies or regime shifts to generate audit narratives, review risk exceptions, and provide qualitative explanations for human compliance officers. This hybrid approach leverages the quantitative strength of ML alongside the qualitative reasoning of LLMs, as demonstrated by the case study in Figure 3.

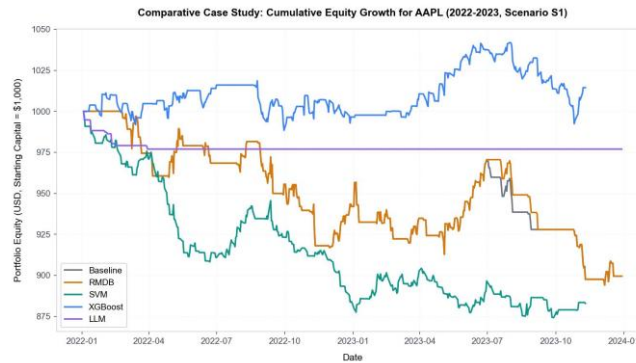


Fig. 3. Cumulative portfolio equity curves comparing Baseline, RMDB, SVM, XGBoost, and LLM Agent on AAPL (2022-2023, Scenario S1).

## 7 Conclusion and Future Work

In this study, we conducted a rigorous empirical evaluation comparing deterministic, supervised machine learning, and large language model agent architectures for algorithmic trading. Our backtesting results across AAPL and spot GOLD demonstrated that the XGBoost supervised walk-forward model consistently achieves superior risk-adjusted returns (Sharpe of 0.204) compared to the Llama-3.3-70B agent under standard slippage, while avoiding the latency and high token costs of generative APIs. Paired bootstrap testing confirmed that the outperformance of traditional ML is statistically significant under stressed slippage scenarios (S2). To govern these trade-offs, we introduced the 'Agentic Minimality Gate,' a deployment ledger that limits LLMs to qualitative auditing and explanation tasks. Future work will explore the integration of multi-modal LLMs that ingest charts directly, as well as the optimization of small, locally hosted, fine-tuned SLMs for risk-escalation tasks.

## 8 References

1. Fama, E.F.: Efficient capital markets: A review of theory and empirical work. *Journal of Finance* 25(2), 383-417 (1970).
2. Murphy, J.J.: *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*. New York Institute of Finance (1999).
3. Cortes, C., Vapnik, V.: Support-vector networks. *Machine Learning* 20(3), 273-297 (1995).
4. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794 (2016).
5. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems*, pp. 4765-4774 (2017).
6. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Xia, F., Le, Q., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35, 24824-24837 (2022).
7. Lopez de Prado, M.: *Advances in financial machine learning*. John Wiley & Sons (2018).
8. Araci, D.: FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063* (2019).
9. Yang, H., Liu, X.Y., Zhong, Q.: FinRL: A deep reinforcement learning library for quantitative finance. *arXiv preprint arXiv:2011.09607* (2020).
10. Efron, B., Tibshirani, R.J.: *An introduction to the bootstrap*. CRC Press (1994).
11. Deng, S., Sakurai, T.: Multiple-source information fusion for stock price prediction. In: *IEEE International Conference on Data Mining*, pp. 833-838 (2014).